

1) Multiple inputs and outputs

A Bitcoin transaction can use multiple inputs and create multiple outputs. Multiple inputs are useful when one old UTXO is not enough to cover the new payment, so several earlier unspent outputs are combined. Multiple outputs are useful because one transaction may pay the receiver, return change to the sender, and possibly create other outputs as well. So Bitcoin transactions are flexible and do not have to be one-input, one-output only.

2) Unspendable output / OP_RETURN

An unspendable output is a Bitcoin output that is intentionally created so that nobody can spend it later. This is commonly done using `OP_RETURN`, which marks the output as provably unspendable and is often used to attach small pieces of data to the blockchain. Since the output is not meant to represent future spendable money, it usually carries zero or negligible value. So `OP_RETURN` is mainly for on-chain data recording, not normal payment transfer.

3) ECDSA signature format

In legacy Bitcoin, digital signatures are created using ECDSA, which stands for Elliptic Curve Digital Signature Algorithm. The signature is not stored as plain text but as a structured sequence of bytes inside the transaction. This signature proves that the spender had the correct private key and authorized the transaction. So ECDSA is the mathematical signature system used in classic Bitcoin transaction authorization.

4) r and s in signature

An ECDSA signature is made of two numbers called **r** and **s**. These two values together form the actual cryptographic signature, and both are required for verification. When people say “the signature” in legacy Bitcoin, they usually mean this `(r, s)` pair packaged into the correct format. So r and s are the core numeric components of the signature.

5) DER encoding idea

DER encoding is the byte-formatting method used to serialize the ECDSA signature values r and s into the transaction. It adds structure such as type markers and length fields so that the signature can be interpreted correctly by the Bitcoin system. The important idea is that the same mathematical signature may sometimes be represented in slightly different but still valid byte forms. That becomes important later when studying transaction malleability.

6) Sighash byte

The sighash byte is an extra value added at the end of the signature format that tells Bitcoin which parts of the transaction were actually hashed and signed. For example, one sighash type may commit to all inputs and outputs, while another may cover a different subset. So the sighash byte defines the exact scope of what the signature is authorizing. It is like a rule that tells the network what this signature is meant to protect.

7) Transaction malleability

Transaction malleability means that the transaction ID of a Bitcoin transaction can sometimes be changed without changing the transaction's real economic effect. In legacy Bitcoin, this happens because the txid is computed from the serialized transaction, and that serialization includes the signature data in `scriptSig`. If someone changes the valid byte encoding of the signature, the txid changes even though the same inputs are spent and the same outputs are created. So the transaction "means" the same thing but ends up with a different ID.

8) Why malleability is a problem

Malleability is a problem because wallets, users, and other protocols often expect the transaction ID to stay stable once the transaction is created. If someone changes the txid by altering the signature encoding, a wallet may not recognize the confirmed transaction as the one it originally sent. It also creates confusion for systems that build later transactions based on earlier txids, especially higher-layer protocols. So malleability breaks reliability even when the actual payment result stays the same.

9) SegWit idea

SegWit, or Segregated Witness, was introduced to fix transaction malleability by separating witness data such as signatures and public keys from the main transaction data used to calculate the txid. In legacy transactions, signatures were inside `scriptSig`, which affected the txid. SegWit moves that witness information out of the txid-calculation area, so changing witness encoding does not change the transaction ID. So the big idea of SegWit is: keep the proof data, but separate it from the transaction identifier.

10) Legacy vs SegWit structure

In legacy Bitcoin transactions, the input contains `scriptSig`, which directly stores the unlocking data like signature and public key. In SegWit transactions, `scriptSig` is empty or reduced, and the unlocking data is moved into a separate witness section. SegWit also adds special marker and flag bytes to indicate the new format. So the main structural difference is that legacy keeps the signature in the main transaction body, while SegWit stores it separately as witness.

11) P2WPKH

P2WPKH stands for **Pay to Witness Public Key Hash**, which is a SegWit version of the old P2PKH idea. Instead of storing a full old-style locking script with obvious opcodes, the output contains a compact witness program that represents a public key hash under SegWit rules. The actual unlocking data — signature and public key — is supplied later in the witness. So P2WPKH is basically a SegWit-style way of saying, “only the owner of this public key hash may spend this output.”

12) Witness

Witness is the part of a SegWit transaction that contains the data proving the right to spend an output, such as the signature and public key. In legacy transactions, this proof lived inside `scriptSig`, but SegWit moves it into a separate witness section. This change makes txids more stable because the witness is no longer part of the main txid calculation. So witness is simply the authorization proof, stored separately.

13) Virtual scriptPubKey

In SegWit transactions like P2WPKH, the on-chain output script looks simpler and does not directly show the old familiar opcodes. But for validation, Bitcoin internally expands it into an equivalent “virtual” script that behaves like a traditional P2PKH locking script. This virtual script is used during execution to check the public key hash and signature correctly. So even though the stored format changes, the underlying ownership logic remains similar.

14) SegWit execution flow

In SegWit execution, `scriptSig` is usually empty, the witness provides the signature and public key, and the previous output’s SegWit program is expanded into a virtual script for checking. The script then verifies that the provided public key matches the expected hash and that the signature is valid. If all checks succeed, the stack ends in `True`, which means the spend is valid. So SegWit execution still proves ownership and authorization, just with the witness separated from the main body.

15) Multisig

A multisig transaction is a Bitcoin transaction that requires more than one signature to spend an output. Instead of trusting one single key, the output can be locked under a policy that says several authorized parties must approve the spend. This is useful for shared wallets, escrow, organizational funds, and stronger security. So multisig means shared control over the ability to move funds.

16) M-of-N

M-of-N describes the rule used in a multisig setup. It means that out of N total authorized public keys, at least M valid signatures are required to spend the output. For example, 2-of-3 means any two out of three possible key holders can authorize the spend. This gives flexibility because not every key must always be present, but one key alone may still be insufficient.

17) P2WSH

P2WSH stands for **Pay to Witness Script Hash** and is the SegWit version of locking value to the hash of a whole script rather than to a single public key hash. It is commonly used for more

complex spending conditions like multisig. The output stores only the hash of the witness script, and the real script is revealed later in the witness when spending. So P2WSH is useful when the spending rule is larger or more complex than a simple one-key ownership rule.

18) Witness script

The witness script is the actual script that defines the spending conditions for a P2WSH output. For example, in a multisig transaction it may say something like “2 of these 3 public keys must sign.” This script is not stored directly in the original output; instead, its hash is stored there, and the real script is revealed in the witness when the output is spent. So the witness script is the hidden rulebook that gets shown at spending time.

19) CHECKMULTISIG bug / OP_0

Bitcoin's `OP_CHECKMULTISIG` has a historical bug where it consumes one extra item from the stack. Because of this, multisig witness data includes a dummy `OP_0` value before the signatures so that the script executes correctly. This extra zero is not meaningful as a real signature; it is only there to satisfy the buggy behavior of the opcode. So `OP_0` in multisig is a workaround for a historical implementation quirk.

20) Verifying transaction inclusion in block

To verify that a transaction really happened, it is not enough to just know its txid. A verifier must confirm that the transaction is actually included in a block, that the block itself is valid, and that the block belongs to the accepted chain. This is important because only then can the transaction be trusted as part of blockchain history. So inclusion verification is really about proving both transaction membership and block legitimacy.

21) Merkle tree

A Merkle tree is a hash-based tree structure used to summarize all transactions in a block. Each transaction is hashed individually, then pairs of hashes are combined and hashed again, and this process continues upward until one final top hash remains. This structure allows efficient

proofs that a specific transaction is included in the block. So a Merkle tree is basically a compact way of organizing many transaction hashes into one verifiable summary.

22) Merkle root

The Merkle root is the top hash of the Merkle tree and acts as a single summary of all the transactions in the block. If even one transaction changes, its hash changes, the tree above it changes, and therefore the Merkle root changes too. This makes the Merkle root a compact integrity fingerprint for the entire transaction set. That is why it is stored in the block header.

23) SPV

SPV stands for **Simplified Payment Verification**. It allows lightweight clients to verify that a transaction is included in a block without downloading the entire block or full blockchain. Instead, the client uses a Merkle proof and the block header to check inclusion. So SPV makes blockchain verification much more practical for lightweight wallets and low-resource devices.

24) $O(n)$ vs $O(\log n)$

If transaction inclusion were checked naively by scanning or recomputing across all transactions in a block, the work could be $O(n)$, meaning it grows linearly with the number of transactions. Merkle trees reduce this to $O(\log n)$ because a verifier only needs the path of sibling hashes from the transaction leaf up to the root. This makes proofs much smaller and faster. So the advantage of Merkle trees is that they turn expensive full-block verification into efficient logarithmic proof checking.